

PYTHON MASTER LIBRARY

Kompletny Słownik, Słowa Kluczowe, Składnia i Dokumentacja Języka Python

Niniejszy dokument stanowi pełne kompendium referencyjne i słownik rdzenia języka Python (v3.10+). Zawiera wszystkie słowa kluczowe, funkcje wbudowane, metody głównych typów danych oraz metody specjalne (magiczne), tworząc komplementarną bibliotekę wiedzy technicznej.

1. Słowa Kluczowe (Keywords)

Słowa kluczowe to zarezerwowane wyrażenia, które posiadają specjalne znaczenie dla kompilatora i interpretera Pythona. Nie mogą być używane jako identyfikatory (nazwy zmiennych czy funkcji).

Słowo kluczowe	Opis działania (Po polsku)	Przykład składni
False	Wartość logiczna fałszu. Wynik operacji porównania lub jawna deklaracja.	<pre>flag = False</pre>
None	Specjalna stała oznaczająca brak wartości, pustkę lub obiekt niezdefiniowany.	<pre>result = None</pre>
True	Wartość logiczna prawdy. Rdzeń wyrażen warunkowych.	<pre>is_active = True</pre>
and	Operator logiczny koniunkcji (I). Zwraca True tylko gdy oba człony są prawdziwe.	<pre>if cond1 and cond2:</pre>
as	Używane do tworzenia aliasów przy importach modułów lub w menedżerach kontekstu.	<pre>import numpy as np with open() as f:</pre>
assert	Instrukcja debugowania. Sprawdza warunek; jeśli fałsz, podnosi AssertionError.	<pre>assert x > 0, "X musi być dodatnie"</pre>
async	Deklaruje asynchroniczny współprogram (coroutine) lub asynchroniczny menedżer kontekstu.	<pre>async def fetch_data():</pre>

Słowo kluczowe	Opis działania (Po polsku)	Przykład składni
await	Wstrzymuje wykonanie współprogramu do momentu otrzymania wyniku operacji asynchronicznej.	<pre>data = await fetch_data()</pre>
break	Natychmiast przerywa działanie najbliższej pętli for lub while .	<pre>while True: break</pre>
class	Służy do definiowania nowej klasy (paradygmat programowania obiektowego).	<pre>class SpaceCraft: pass</pre>
continue	Pomija resztę instrukcji w bieżącej iteracji pętli i przechodzi do następnej.	<pre>for i in range(5): continue</pre>
def	Służy do definiowania funkcji lub metod wewnątrz klas.	<pre>def calculate(x, y):</pre>
del	Usuwa referencję do obiektu, element listy pod indeksem lub klucz ze słownika.	<pre>del my_list[0] del variable</pre>
elif	"Else if" — warunek opcjonalny sprawdzany, gdy poprzedni warunek if był fałszywy.	<pre>if x == 1: elif x == 2:</pre>
else	Blok domyślny dla instrukcji warunkowych oraz pętli (gdy pętla zakończy się bez break).	<pre>if cond: else:</pre>
except	Przechwytuje i obsługuje określone wyjątki w strukturze try-except .	<pre>try: except ValueError:</pre>

Słowo kluczowe	Opis działania (Po polsku)	Przykład składni
finally	Blok kodu wykonywany zawsze, bez względu na to, czy wyjątek wystąpił, czy nie.	<pre>try: finally: clean_up()</pre>
for	Pętla iteracyjna przechodząca po elementach sekwencji (listy, krotki, generatory).	<pre>for item in items:</pre>
from	Określa moduł źródłowy, z którego importujemy wybrane elementy.	<pre>from math import sqrt</pre>
global	Deklaruje, że zmienna wewnątrz funkcji odnosi się do zasięgu globalnego (modułu).	<pre>def update(): global counter</pre>
if	Podstawowa instrukcja warunkowa decydująca o wykonaniu bloku kodu.	<pre>if temperature > 100:</pre>
import	Dołącza zewnętrzny moduł lub bibliotekę do bieżącego skryptu.	<pre>import os, sys</pre>
in	Operator sprawdzający zawieranie się (przynależność) lub element składni pętli for .	<pre>if "a" in "space":</pre>
is	Operator tożsamości. Sprawdza, czy dwie zmienne wskazują na ten sam obiekt w pamięci.	<pre>if x is None:</pre>
lambda	Tworzy anonimową, jednowierszową funkcję wyrażeniową.	<pre>square = lambda x: x**2</pre>
nonlocal	Deklaruje zmienną z domknięcia (zewnętrznej funkcji, ale nie globalnej).	<pre>def outer(): nonlocal x</pre>

Słowo kluczowe	Opis działania (Po polsku)	Przykład składni
not	Operator logiczny negacji. Odwraca wartość logiczną wyrażenia.	<pre>if not is_ready:</pre>
or	Operator logiczny alternatywy (LUB). True, jeśli przynajmniej jeden człon jest prawdziwy.	<pre>if cond1 or cond2:</pre>
pass	Pusta instrukcja (placeholder). Nie robi nic, zachowuje poprawność syntaktyczną struktury.	<pre>def todo_function(): pass</pre>
raise	Jawne wywołanie (podniesienie) określonego błędu/wyjątku.	<pre>raise KeyError("Brak klucza")</pre>
return	Kończy działanie funkcji i opcjonalnie zwraca określoną wartość.	<pre>return total_cost</pre>
try	Inicjuje blok kodu monitorowany pod kątem wystąpienia błędów.	<pre>try: process()</pre>
while	Pętla wykonująca blok kodu tak długo, jak jej warunek pozostaje prawdziwy.	<pre>while x < 10:</pre>
with	Uruchamia kod wewnątrz menedżera kontekstu (zapewnia automatyczne zwalnianie zasobów).	<pre>with open('file.txt') as f:</pre>
yield	Zwraca wartość z generatora i wstrzymuje stan funkcji do następnego wywołania.	<pre>yield element</pre>
match	Wprowadzone w Python 3.10 strukturalne dopasowanie wzorców (odpowiednik switch).	<pre>match status_code:</pre>

Słowo kluczowe	Opis działania (Po polsku)	Przykład składni
<code>case</code>	Definiuje konkretny wzorzec dopasowania wewnątrz instrukcji <code>match</code> .	<pre>case 200: return "OK"</pre>

2. Funkcje Wbudowane (Built-in Functions)

Funkcje dostępne globalnie w każdym skrypcie Pythona bez konieczności importowania jakichkolwiek modułów.

Funkcja	Opis zastosowania i parametrów	Krótki przykład
<code>abs(x)</code>	Zwraca wartość bezwzględną liczby.	<code>abs(-5) # 5</code>
<code>all(iterable)</code>	Zwraca True, jeśli wszystkie elementy są prawdziwe.	<code>all([True, 1]) # True</code>
<code>any(iterable)</code>	Zwraca True, jeśli co najmniej jeden element jest prawdziwy.	<code>any([False, 1]) # True</code>
<code>bin(x)</code>	Konwertuje liczbę całkowitą na ciąg binarny (z prefiksem 0b).	<code>bin(4) # '0b100'</code>
<code>bool(x)</code>	Konwertuje wartość na typ logiczny Boolean.	<code>bool(0) # False</code>
<code>breakpoint()</code>	Uruchamia wbudowany debugger (pdb) w miejscu wywołania.	<code>breakpoint()</code>
<code>bytes(x)</code>	Tworzy niezmienny obiekt bajtowy.	<code>bytes([65, 66]) # b'AB'</code>
<code>callable(obj)</code>	Zwraca True, jeśli obiekt może być wywołany jak funkcja.	<code>callable(print) # True</code>
<code>chr(i)</code>	Zwraca znak reprezentujący podany kod Unicode.	<code>chr(97) # 'a'</code>
<code>classmethod(f)</code>	Dekorator przekształcający metodę w metodę klasy.	<code>@classmethod</code>
<code>complex(r, i)</code>	Tworzy liczbę zespoloną.	<code>complex(2, 3) # (2+3j)</code>
<code>delattr(obj, name)</code>	Usuwa atrybut z obiektu o podanej nazwie str.	<code>delattr(car, 'color')</code>
<code>dict(**kw)</code>	Tworzy i inicjalizuje nowy słownik.	<code>dict(a=1, b=2)</code>
<code>dir(obj)</code>	Zwraca listę prawidłowych atrybutów i metod obiektu.	<code>dir([])</code>

Funkcja	Opis zastosowania i parametrów	Krótki przykład
<code>divmod(a, b)</code>	Zwraca krotkę zawierającą iloraz i resztę z dzielenia.	<code>divmod(7, 3) # (2, 1)</code>
<code>enumerate(it)</code>	Zwraca obiekt iterowalny z parą (indeks, element).	<code>enumerate(['a'])</code>
<code>eval(expr)</code>	Wykonuje dynamicznie wyrażenie Pythona przekazane jako str.	<code>eval("2 + 2") # 4</code>
<code>exec(object)</code>	Dynamicznie wykonuje pełny kod źródłowy Pythona.	<code>exec("x = 5")</code>
<code>filter(f, it)</code>	Filtruje elementy kolekcji na podstawie funkcji zwracającej True/False.	<code>filter(lambda x: x>0, 1)</code>
<code>float(x)</code>	Konwertuje liczbę lub ciąg znaków na typ zmiennoprzecinkowy.	<code>float("3.14") # 3.14</code>
<code>format(v, spec)</code>	Formatuje wartość według określonej specyfikacji.	<code>format(0.5, '%') # '50.00000%'</code>
<code>frozenset(it)</code>	Tworzy niezmienny (zamrożony) zbiór unikalnych wartości.	<code>frozenset([1, 2])</code>
<code>getattr(o, n)</code>	Pobiera wartość atrybutu obiektu dynamicznie po nazwie string.	<code>getattr(obj, 'x')</code>
<code>globals()</code>	Zwraca słownik reprezentujący aktualną globalną tablicę symboli.	<code>globals()</code>
<code>hasattr(o, n)</code>	Zwraca True, jeśli obiekt posiada atrybut o danej nazwie.	<code>hasattr(obj, 'x')</code>
<code>hash(obj)</code>	Zwraca wartość skrótu (hash) obiektu (dla obiektów hashable).	<code>hash("abc")</code>
<code>hex(x)</code>	Konwertuje liczbę na ciąg szesnastkowy (prefiks 0x).	<code>hex(255) # '0xff'</code>
<code>id(obj)</code>	Zwraca unikalny identyfikator tożsamości obiektu (adres pamięci).	<code>id(x)</code>
<code>input(prompt)</code>	Pobiera ciąg znaków wprowadzony przez użytkownika z konsoli.	<code>name = input("Podaj imię")</code>
<code>int(x, base)</code>	Konwertuje wartość na liczbę całkowitą (można podać bazę systemu).	<code>int("10", 2) # 2</code>
<code>isinstance(o, c)</code>	Sprawdza, czy obiekt jest instancją podanej klasy lub krotki klas.	<code>isinstance(5, int) # True</code>
<code>issubclass(c, b)</code>	Sprawdza, czy klasa jest podklasą (dziedziczy po) innej klasy.	<code>issubclass(Dog, Animal)</code>

Funkcja	Opis zastosowania i parametrów	Krótki przykład
<code>iter(obj)</code>	Zwraca obiekt iteratora dla podanej kolekcji.	<code>it = iter([1, 2])</code>
<code>len(s)</code>	Zwraca liczbę elementów (długość) kolekcji lub ciągu znaków.	<code>len("Akasha") # 6</code>
<code>list(it)</code>	Tworzy modyfikowalną strukturę listy z obiektu iterowalnego.	<code>list((1, 2)) # [1, 2]</code>
<code>locals()</code>	Zwraca słownik zawierający aktualne zmienne lokalne.	<code>locals()</code>
<code>map(func, it)</code>	Aplikuje podaną funkcję do każdego elementu sekwencji.	<code>map(str.upper, ['a'])</code>
<code>max(iterable)</code>	Zwraca największy element z podanej kolekcji.	<code>max([3, 9, 1]) # 9</code>
<code>min(iterable)</code>	Zwraca najmniejszy element z podanej kolekcji.	<code>min([3, 9, 1]) # 1</code>
<code>next(iterator)</code>	Zwraca kolejny element z iteratora.	<code>next(it)</code>
<code>object()</code>	Zwraca nową, pustą bazową instancję typu object.	<code>obj = object()</code>
<code>oct(x)</code>	Konwertuje liczbę na system ósemkowy (z prefiksem 0o).	<code>oct(8) # '0o10'</code>
<code>open(file, mode)</code>	Otwiera plik i zwraca odpowiedni obiekt pliku (stream).	<code>open('data.log', 'r')</code>
<code>ord(c)</code>	Zwraca liczbę całkowitą reprezentującą kod Unicode znaku.	<code>ord('a') # 97</code>
<code>pow(base, exp)</code>	Potęgowanie liczby (odpowiednik <code>base ** exp</code>).	<code>pow(2, 3) # 8</code>
<code>print(*objects)</code>	Drukuje obiekty na standardowe wyjście (konsolę).	<code>print("Hello", end="")</code>
<code>property()</code>	Używane jako dekorator do definiowania właściwości get/set/del.	<code>@property</code>
<code>range(stop)</code>	Generuje sekwencję liczb od 0 do stop-1.	<code>range(0, 10, 2)</code>
<code>repr(obj)</code>	Zwraca oficjalną tekstową reprezentację obiektu zdatną do odtworzenia.	<code>repr("a") # "'a'"</code>
<code>reversed(seq)</code>	Zwraca odwrócony iterator dla danej sekwencji.	<code>reversed([1, 2, 3])</code>
<code>round(n, ndig)</code>	Zaokrągla liczbę do określonej liczby miejsc po przecinku.	<code>round(3.14159, 2) # 3.14</code>

Funkcja	Opis zastosowania i parametrów	Krótki przykład
<code>set(iterable)</code>	Tworzy mutowalny zbiór unikalnych wartości.	<code>set([1, 1, 2]) # {1, 2}</code>
<code>setattr(o, n, v)</code>	Ustawia wartość określonego atrybutu obiektu dynamicznie.	<code>setattr(obj, 'val', 42)</code>
<code>sorted(iterable)</code>	Zwraca nową, posortowaną listę z elementów kolekcji.	<code>sorted([3,1,2]) # [1,2,3]</code>
<code>staticmethod(f)</code>	Dekorator przekształcający metodę w metodę statyczną.	<code>@staticmethod</code>
<code>str(object)</code>	Konwertuje obiekt na reprezentację tekstową (czytelną dla ludzi).	<code>str(42) # '42'</code>
<code>sum(iterable)</code>	Sumuje wszystkie elementy numeryczne podanej kolekcji.	<code>sum([1, 2, 3]) # 6</code>
<code>super()</code>	Zwraca obiekt proxy umożliwiający dostęp do metod klasy nadrzędnej.	<code>super().__init__()</code>
<code>tuple(iterable)</code>	Tworzy niezmienną krotkę (tuple) z obiektu iterowalnego.	<code>tuple([1,2]) # (1,2)</code>
<code>type(object)</code>	Zwraca typ obiektu lub tworzy nowy obiekt klasy dynamicznie.	<code>type(5) #</code>
<code>zip(*iterables)</code>	Agreguje elementy z wielu kolekcji w krotki pozycyjne.	<code>zip([1,2], ['a', 'b'])</code>

3. Metody Głównych Typów Danych (Built-in Types Methods)

3.1. Metody Typu String (Ciągi Znaków - `str`)

Wszystkie metody zwracają nową wartość, ponieważ ciągi znaków w Pythonie są niezmiennie (immutable).

Metoda	Opis działania
<code>.upper()</code>	Zmienia wszystkie litery na wielkie.
<code>.lower()</code>	Zmienia wszystkie litery na małe.
<code>.split(sep)</code>	Dzieli ciąg na listę podciągów według separatora <code>sep</code> .
<code>.join(iterable)</code>	Łączy elementy obiektu iterowalnego w jeden string za pomocą łącznika.
<code>.strip()</code>	Usuwa wiodące i końcowe białe znaki (oraz spacje).

Metoda	Opis działania
<code>.replace(old, new)</code>	Zastępuje wszystkie wystąpienia frazy old frazą new .
<code>.startswith(prefix)</code>	Zwraca True, jeśli ciąg zaczyna się od podanego prefiksu.
<code>.endswith(suffix)</code>	Zwraca True, jeśli ciąg kończy się danym sufiksem.
<code>.find(sub)</code>	Zwraca najniższy indeks, na którym znajduje się podciąg sub (lub -1).
<code>.index(sub)</code>	To samo co <code>.find()</code> , ale rzuca ValueError, gdy nie znajdzie podciagu.
<code>.isdigit()</code>	Zwraca True, jeśli ciąg składa się wyłącznie z cyfr.
<code>.isalpha()</code>	Zwraca True, jeśli ciąg zawiera tylko litery alfabetu.
<code>.isalnum()</code>	Zwraca True, jeśli ciąg zawiera wyłącznie znaki alfanumeryczne (litery i cyfry).
<code>.count(sub)</code>	Zwraca liczbę bezkolizyjnych wystąpień podciagu sub .

3.2. Metody Typu List (Listy - **list**)

Listy są modyfikowalne in-place. Metody modyfikują oryginalną strukturę danych.

Metoda	Opis działania
<code>.append(item)</code>	Dodaje element na sam koniec listy.
<code>.extend(iterable)</code>	Rozszerza listę, dodając wszystkie elementy z przekazanej kolekcji.
<code>.insert(index, item)</code>	Wstawia element na dokładnie określone miejsce (indeks).
<code>.remove(item)</code>	Usuwa pierwsze napotkane wystąpienie podanego elementu. Rzuca ValueError przy braku.
<code>.pop(index)</code>	Usuwa i zwraca element z podanego indeksu (domyślnie ostatni).
<code>.clear()</code>	Usuwa absolutnie wszystkie elementy z listy, pozostawiając ją pustą.
<code>.index(item)</code>	Zwraca indeks pierwszego wystąpienia elementu.
<code>.count(item)</code>	Zwraca liczbę wystąpień określonego elementu w liście.
<code>.sort(reverse, key)</code>	Sortuje listę na miejscu (modyfikacja in-place).
<code>.reverse()</code>	Odwraca kolejność elementów listy na miejscu.
<code>.copy()</code>	Zwraca płytką kopię (shallow copy) listy.

3.3. Metody Typu Dictionary (Słowniki - `dict`)

Metoda	Opis działania
<code>.clear()</code>	Czyści cały słownik ze wszystkich par klucz-wartość.
<code>.get(key, default)</code>	Zwraca wartość dla klucza; jeśli klucz nie istnieje, zwraca wartość domyślną zamiast błędu.
<code>.items()</code>	Zwraca dynamiczny widok par typu krotka (klucz, wartość) .
<code>.keys()</code>	Zwraca dynamiczny widok zawierający wyłącznie klucze słownika.
<code>.values()</code>	Zwraca dynamiczny widok zawierający wyłącznie wartości słownika.
<code>.pop(key, default)</code>	Usuwa wskazany klucz i zwraca przypisaną mu wartość.
<code>.popitem()</code>	Usuwa i zwraca ostatnią parę (klucz, wartość) w porządku LIFO.
<code>.update(other)</code>	Aktualizuje słownik par wartościami z innego słownika lub obiektu iterowalnego.
<code>.setdefault(k, d)</code>	Zwraca wartość klucza; jeśli klucz nie istnieje, wkłada go do słownika z wartością d.

3.4. Metody Typu Set (Zbiory - `set`)

Metoda	Opis działania
<code>.add(element)</code>	Dodaje element do zbioru (jeśli go jeszcze nie ma).
<code>.remove(element)</code>	Usuwa element ze zbioru. Zgłasza <code>KeyError</code> , jeśli elementu nie ma.
<code>.discard(element)</code>	Usuwa element ze zbioru, ale nie generuje błędu, jeśli element nie istnieje.
<code>.union(*sets)</code>	Zwraca nowy zbiór będący sumą wielośćciową zbiorów.
<code>.intersection(*sets)</code>	Zwraca część wspólną (iloczyn) zbiorów.
<code>.difference(*sets)</code>	Zwraca różnicę zbiorów (elementy obecne tylko w zbiorze wyjściowym).
<code>.update(*sets)</code>	Modyfikuje zbiór, dodając do niego elementy ze wskazanych zbiorów.

4. Metody Specjalne / Magiczne (Magic Methods)

Metody rozpoczynające i kończące się podwójnym podkreśleniem `__` (Dunder methods). Umożliwiają przeciążanie operatorów i integrację własnych obiektów klasowych z ekosystemem Pythona.

Metoda Magiczna	Przeznaczenie i wywołanie	Powiązany operator / akcja
<code>__init__(self, ...)</code>	Inicjalizator nowo utworzonego obiektu klasy (konstruktor).	Wywoływane przy tworzeniu instancji: <code>Obj()</code>
<code>__new__(cls, ...)</code>	Rzeczywisty konstruktor tworzący instancję obiektu przed init.	Wywoływane automatycznie przez interpreter.
<code>__str__(self)</code>	Zwraca czytelną, przyjazną użytkownikowi reprezentację str obiektu.	Wykonywane przez <code>str(obj)</code> lub <code>print(obj)</code>
<code>__repr__(self)</code>	Zwraca oficjalną, jednoznaczną techniczną reprezentację obiektu.	Wykonywane przez funkcję <code>repr(obj)</code> lub w powłoce.
<code>__len__(self)</code>	Zwraca całkowitą długość obiektu (liczbę elementów).	Wykonywane przez <code>len(obj)</code>
<code>__getitem__(self, k)</code>	Obsługuje odczyt elementu z obiektu przy pomocy indeksu/kłucza.	Wywołanie przez nawiasy kwadratowe: <code>obj[k]</code>
<code>__setitem__(self, k, v)</code>	Obsługuje przypisanie wartości pod dany klucz lub indeks.	Wywołanie przez przypisanie: <code>obj[k] = v</code>
<code>__delitem__(self, k)</code>	Obsługuje usuwanie elementu pod kluczem/ indeksem.	Wywołanie: <code>del obj[k]</code>
<code>__add__(self, other)</code>	Przeciążenie operatora dodawania.	Operator <code>+</code>
<code>__sub__(self, other)</code>	Przeciążenie operatora odejmowania.	Operator <code>-</code>
<code>__mul__(self, other)</code>	Przeciążenie operatora mnożenia.	Operator <code>*</code>
<code>__truediv__(self, o)</code>	Przeciążenie operatora standardowego dzielenia zmiennoprzecinkowego.	Operator <code>/</code>
<code>__eq__(self, other)</code>	Sprawdza relację równości obiektów.	Operator <code>==</code>
<code>__lt__(self, other)</code>	Sprawdza relację "mniejszy niż".	Operator <code><</code>
<code>__contains__(self, item)</code>	Sprawdza przynależność elementu do struktury obiektu.	Operator przynależności: <code>item in obj</code>
<code>__enter__(self)</code>	Inicjuje wejście do menedżera kontekstu block.	Początek instrukcji <code>with obj:</code>
<code>__exit__(self, *args)</code>	Obsługuje wyjście z menedżera kontekstu i ewentualne wyjątki.	Koniec instrukcji lub wyjście z bloku <code>with</code>
<code>__call__(self, ...)</code>	Pozwala na wywołanie instancji obiektu tak, jakby była funkcją.	Wywołanie: <code>instancja()</code>

5. Główne Wyjątki Wbudowane (Built-in Exceptions)

Hierarchia podstawowych wyjątków i błędów, które mogą zostać wygenerowane w trakcie uruchamiania kodu.

- **BaseException** — Główny korzeń wszystkich wyjątków Pythona.
- **Exception** — Klasa bazowa dla wszystkich niekrytycznych wyjątków (zalecana do dziedziczenia).
- **TypeError** — Podnoszony, gdy operacja lub funkcja jest stosowana do obiektu niewłaściwego typu.
- **ValueError** — Występuje, gdy funkcja otrzymuje argument właściwego typu, ale o niepoprawnej wartości.
- **NameError** — Podnoszony, gdy lokalna lub globalna nazwa zmiennej nie zostanie znaleziona.
- **IndexError** — Wywoływany, gdy indeks sekwencji (np. listy) jest poza dopuszczalnym zakresem.
- **KeyError** — Podnoszony, gdy klucz słownika nie zostanie znaleziony w zestawie istniejących kluczy.
- **AttributeError** — Występuje, gdy odniesienie do atrybutu lub przypisanie się nie powiedzie.
- **FileNotFoundError** — Podnoszony, gdy żądany plik lub katalog nie istnieje w systemie.
- **ZeroDivisionError** — Wywoływany, gdy drugim argumentem operacji dzielenia lub modulo jest zero.